

ReSeek: A Self-Correcting Framework for Search Agents with Instructive Rewards

Shiyu Li¹ Yifan Wang^{1,2} Peiming Li¹ Zheng Wei¹ Yang Tang¹

[Homepage](#) [Code](#) [Leaderboard](#) [Dataset](#) [Model](#)

Abstract

Search agents powered by Large Language Models (LLMs) have demonstrated significant potential in tackling knowledge-intensive tasks. Reinforcement learning (RL) has emerged as a powerful paradigm for training these agents to perform complex, multi-step reasoning. However, prior RL-based methods often rely on sparse or rule-based rewards, which can lead agents to commit to suboptimal or erroneous reasoning paths without the ability to recover. To address these limitations, we propose **ReSeek**, a self-correcting framework enabling search agents to recover from erroneous search paths during an episode. By invoking a special **JUDGE** action, the agent can judge the information and re-plan its search strategy. To guide this process, we design a dense, instructive process reward function, which combines an answer correctness reward for task completion with a self-correction reward that trains the agent to judge the utility of retrieved information. Additionally, to mitigate the risk of data contamination in existing datasets, we introduce **FictionalHot**, a contamination-resistant benchmark requiring complex reasoning. Experiments show ReSeek significantly outperforms SOTA baselines in task success and path faithfulness. Our code and dataset are available at <https://github.com/TencentBAC/ReSeek>.

1. Introduction

Large Language Models (LLMs) (Brown et al., 2020; OpenAI, 2022; Ouyang et al., 2022; Guo et al., 2025; Yang

Project Leader: Yang Tang. ¹Basic Algorithm Center, PCG, Tencent, Shenzhen, China. ²Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China. Correspondence to: Yang Tang <ethanntang@tencent.com>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

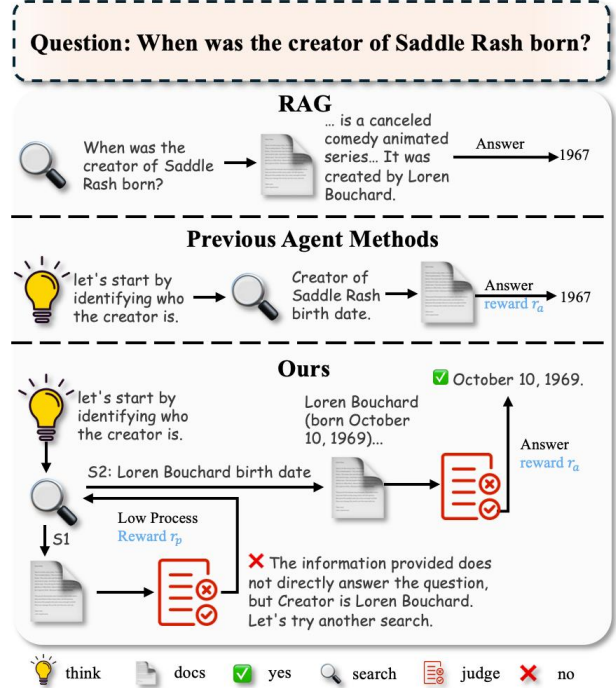


Figure 1. A comparison of reasoning processes on a multi-hop question about an obscure entity. Standard RAG fails as it cannot perform sequential reasoning. Vanilla agent reasons sequentially but gets stuck on its initial path. In contrast, our agent demonstrates robust self-correction: it uses a low process reward (r_p) to identify the unproductive intermediate step, triggers a JUDGE action to revise its strategy, and successfully navigates to the correct answer. The full trace for this example is provided in Appendix A.11.2.

et al., 2025) have demonstrated unprecedented capabilities in natural language understanding and generation, yet they are inherently limited by their static, pre-trained knowledge, which can be outdated or lead to factual hallucinations (Borgeaud et al., 2022; Zhao et al., 2024; Maleki et al., 2024; Zhang et al., 2025b). Search-augmented agents, which empower LLMs to interact with external tools like search engines, have emerged as a powerful paradigm to overcome these limitations (Li et al., 2025c; Zheng et al., 2025; Jin et al., 2025a; Luo et al., 2025). By dynamically retrieving and reasoning over up-to-date information, these agents can tackle complex tasks that are beyond the reach

of standalone LLMs.

Despite their promise, prevailing methods often falter in complex reasoning scenarios. Early approaches like Retrieval-Augmented Generation (RAG) are limited to a single retrieve-then-generate cycle, lacking the capability for error correction. More advanced RL-trained agents learn policies over action sequences, but their sparse or rule-based rewards (e.g., final answer correctness) provide insufficient guidance for intermediate steps. Consequently, an early mistake (such as a misleading search query) can cascade into an incorrect answer, as agents lack the ability to self-assess and recover. As shown in Figure 1, both RAG and agent methods (Jin et al., 2025a) query for “creator of Saddle Rash.” The retrieved documents center almost on describing the show itself and contain no information about the director’s birth date, ultimately leading to an unsuccessful response.

To address these limitations, we propose ReSeek, a self-correcting framework for training search agents, centered on a special JUDGE action. This action enables the agent to pause, evaluate the utility of retrieved information, and dynamically adapt its strategy when the current path proves unproductive. As illustrated in Figure 1, the initial search revealed the creator’s name (Loren Bouchard) but not the birth date. The JUDGE action identified this gap and evaluated the retrieved information as useful yet insufficient. Consequently, the agent adapted its strategy by formulating a follow-up query “Loren Bouchard birth date” and successfully retrieved the correct answer. This entire process is guided by a **dense and instructive reward function** that combines an answer correctness reward for task completion with a self-correction reward for judging evidence utility, providing fine-grained, step-by-step feedback.

Another challenge lies in evaluation. Many existing datasets for knowledge-intensive tasks are at risk of data contamination. Their contents may overlap with LLM training corpora, causing high performance to reflect memorization rather than genuine reasoning. To address this, we first introduce **FictionalHot**, a new benchmark composed of questions about fictional entities. Its design inherently mitigates contamination risk, forcing agents to rely on search results. Our contributions are summarized as follows:

- We propose ReSeek, a novel framework enabling search agents to self-correct and dynamically adapt their search strategy.
- We design a dense, process-based reward function that combines an answer correctness reward with a self-correction reward for judging evidence utility.
- We introduce FictionalHot, a contamination-resistant benchmark for fair and challenging evaluation of search agents’ reasoning abilities.

- Experiments show ReSeek outperforms SOTA baselines in performance and reasoning path faithfulness.

2. Related Work

2.1. RAG and Search Agents

To address knowledge limitations and hallucinations of LLMs (Zhang et al., 2023; Ji et al., 2023; Bang et al., 2023), research has focused on integrating external knowledge. RAG frameworks (Lewis et al., 2020; Guu et al., 2020; Karpukhin et al., 2020) enhance LLM responses by retrieving relevant documents, but their single-step cycle limits efficacy on multi-step reasoning tasks (Jiang et al., 2023; Asai et al., 2023). To address this limitation, advanced Search Agents have been developed (Shinn et al., 2023), which decompose complex tasks by interleaving search and reasoning steps (Yao et al., 2022). Works such as WebThinker (Li et al., 2025c), DeepResearcher (Zheng et al., 2025; Jin et al., 2025a), Search-o1 (Li et al., 2025b), and ZeroSearch (Sun et al., 2025) significantly improve performance on knowledge-intensive tasks by empowering LLMs to plan and execute multi-step strategies. For contamination-aware evaluation, MDBench (Peper et al., 2025) constructs a synthetic multi-document reasoning benchmark from structured knowledge. Our FictionalHot reduces contamination at the source via fictional entity replacement.

2.2. Search with Reinforcement Learning

By modeling the search process as a Markov Decision Process, Reinforcement Learning (RL) can optimize an agent’s policy to maximize cumulative rewards. Recent works such as Search-R1 (Jin et al., 2025a), ToolRL (Qian et al., 2025), and others (Mai et al., 2025; Zeng et al., 2023) have successfully applied RL to train LLMs to use search tools. A key challenge in this paradigm, however, lies in the design of the reward function. Many existing methods rely on sparse, outcome-based rewards (e.g., correctness of the answer) (Jin et al., 2025a; Mai et al., 2025). While straightforward to implement, such rewards provide poor credit assignment for intermediate steps, offering little guidance for navigating complex reasoning paths (Ouyang et al., 2022). Consequently, research has shifted towards denser guidance through process supervision and self-correction mechanisms (Chen et al., 2023). For instance, Backtracking Correction (Feng et al., 2025) refines the reasoning chain in a post-hoc manner, optimizing backwards from a completed trajectory. S2R (Ma et al., 2025) trains self-verification for mathematical reasoning. AgentPRM (Xi et al., 2026) trains a separate process reward model for general interactive agents, while ReSeek uses a search-specific reranker reward and embeds JUDGE directly as an internal action. In contrast to these prior directions, ReSeek enables dynamic self-correction within search-augmented QA trajectories.

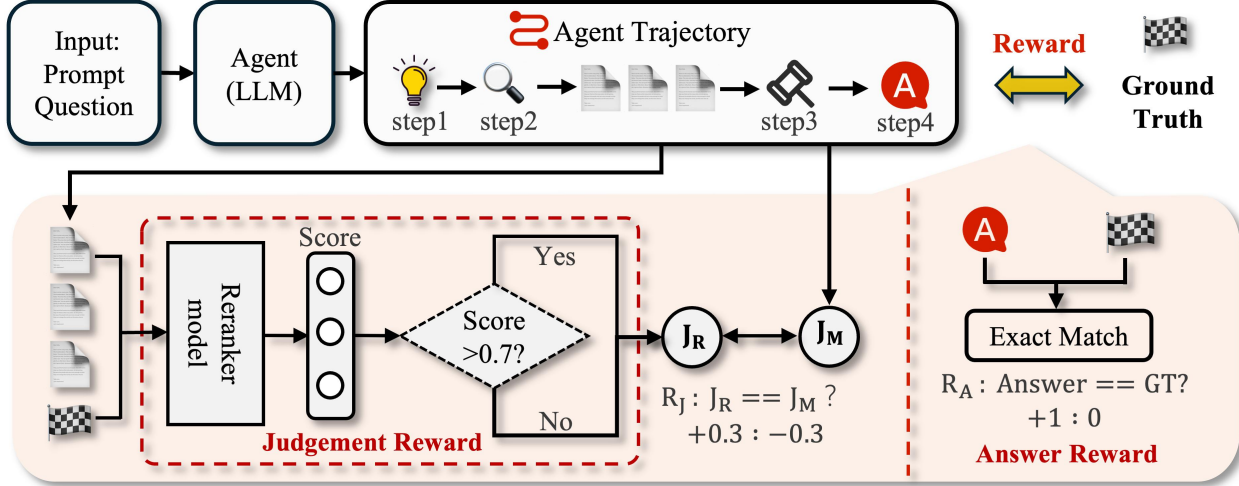


Figure 2. **Training the agent’s self-evaluation capability.** We train the agent via policy optimization to master the JUDGE action. A reward signal is generated by comparing the agent’s judgment against an “ideal” one, which is determined by the rerank score between the current search observation and the GT answer. This reward guides the policy to learn effective self-correction.

3. Methodology

In this section, we present a framework for training LLM agents on complex tasks. Our approach is centered on enhancing the agent’s decision-making capabilities through a novel reinforcement learning paradigm. We begin by formally defining the problem.

3.1. Problem Formulation

We formulate the agent’s interaction as a Markov Decision Process (MDP). The agent is represented by a policy π_θ parameterized by an LLM. At each step t , given the state s_t , the policy generates an action $a_t \in \mathcal{A}$.

To enable self-correction, we define the action space $\mathcal{A}$ to encompass not only standard tool invocations but also evaluation steps (detailed as the JUDGE mechanism in Sec. 3.2). We optimize π_θ against a reference policy π_{ref} using a Group Relative Policy Optimization (GRPO) based objective:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [R(x, y)] - \beta D_{KL}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)] \quad (1)$$

Here, y represents the generated trajectory for a problem instance x from dataset $\mathcal{D}$. $R(x, y)$ aggregates rewards from task completion and intermediate evaluations, while the β -weighted KL term regularizes π_θ towards π_{ref} for stability.

3.2. JUDGE Action

A core challenge for search agents is reliability; a single error can derail an entire task. To operate robustly, agents require meta-cognitive awareness to assess the utility of intermediate steps. To achieve this, we introduce the JUDGE

action, a mechanism designed to serve as the agent’s instrument for introspection. The JUDGE action converts the agent’s reasoning from a static, linear chain into a *dynamic self-correction loop*.

Instead of physically removing uninformative steps (which might lead to repeating the same errors), we employ a **soft removal** strategy. The agent executes a judgment action to generate a label j_t for the current observation o_t . This label is appended to the interaction history, serving as a signal that guides the policy to semantically attend to or disregard the preceding information.

We formalize this process as follows. Let τ_{t-1} denote the interaction trajectory up to step $t-1$. The policy conditions its next action a_{t+1} on the updated context $\mathcal{C}_t$:

$$a_{t+1} \sim \pi(\cdot | \mathcal{C}_t) \quad \text{where} \quad \mathcal{C}_t = \tau_{t-1} \oplus a_t \oplus o_t \oplus j_t \quad (2)$$

Here, $\oplus$ denotes concatenation. In this formulation, the judgment $j_t \in \{\text{'good'}, \text{'bad'}\}$ acts as a **cognitive filter**. A positive judgment reinforces the evidence o_t , while a negative judgment marks that o_t is unhelpful, effectively “blocking” the error path to prevent repeated mistakes.

3.3. Reward Function

Our reward function is composed of two distinct signals: a rule-based reward for task completion and a dense intermediate reward for self-correction.

First, we define the standard objective for the task. The Correctness Reward is a rule-based signal assigned only after the final ANSWER action. It is based on an Exact

Table 1. Self-corrective agent prompt. Following each search, the model executes `<judge>` to determine if it should search again or answer. The Question placeholder is replaced at runtime with the current query.

Answer the given question step by step. Instructions:

1. First, conduct reasoning inside `<think>` and `</think>` tags whenever you receive new information.
2. If you need external knowledge, you can search using `<search>` query `</search>`.
3. When you receive search results, evaluate their usefulness and put your judgment inside `<judge>` Yes `</judge>` or `<judge>` No `</judge>` tags.
4. Based on your judgment, follow these strict rules:
 - a. If the information is useful AND you now have sufficient information to provide a complete final answer, proceed directly to step 5.
 - b. If the information is useful BUT you still need more details, you MUST search again with `<search>` ... `</search>`
 - c. If the information is not useful, you MUST search again with `<search>` ... `</search>`. You MUST NOT provide an answer in this case.
5. Provide your final answer in `<answer>` ... `</answer>` tags. The `<answer>` tag marks the end of the task. After providing the `<answer>`, you MUST stop and generate no further text.

Question: [question]

Match (EM) with the ground truth A_g and prediction A_p :

$$R_{\text{answer}} = EM(A_p, A_g) \quad (3)$$

While this provides the ultimate objective, relying solely on this sparse and delayed signal makes it difficult for the agent to learn the nuanced skill of step-by-step self-assessment.

To address the sparsity of the terminal reward, we introduce a dense, intermediate reward signal specifically designed to cultivate the agent’s self-correction faculty. The core idea is to reward the agent for making judgments that align with an objective evaluation of the retrieved information.

We first establish an objective “Ideal Judgment,” j_t^* , for each observation. This ideal is based on the principle that retrieved information (observation) is useful if it exhibits high semantic similarity to the ground truth. We formalize this as:

$$j_t^* = \begin{cases} \text{‘Yes’} & \text{if } \text{score}(o_t, \text{gt}) > 0.7 \\ \text{‘No’} & \text{otherwise} \end{cases} \quad (4)$$

Here, o_t represents the retrieved information (observation) at step t . The score function is implemented using `bgeranker-large` (Xiao et al., 2023), which computes the semantic relevance between the observation o_t and the ground-truth answer GT. The threshold of 0.7 was determined empirically (Sec 4.3), as it provided a separation between useful and non-useful information.

With the ideal judgment established, we define the self-correction reward R_{judge} which is given whenever the agent performs a JUDGE action:

$$R_{\text{judge}}(j_t, j_t^*) = \begin{cases} R_{\text{match}} & \text{if } j_t = j_t^* \\ R_{\text{mismatch}} & \text{if } j_t \neq j_t^* \end{cases} \quad (5)$$

Here, j_t is the agent’s judgment. R_{match} and R_{mismatch} is the reward value. The R_{match} provides a reward of 0.3 for both types of correct judgments: correctly identifying useful information ($j_t = j_t^* = \text{Yes}$) and discarding useless information ($j_t = j_t^* = \text{No}$). The R_{mismatch} implements asymmetric penalties: correctly accepting useless information incurs a large penalty of -0.6, while discarding useful information incurs a smaller penalty of -0.3.

Finally, the complete training objective is to maximize the total reward over the trajectory τ :

$$R(\tau) = \sum_{t=1}^T \gamma^{t-1} r_t \quad (6)$$

where r_t is defined conditionally based on the temporal stage of the process, specifically corresponding to the judgment reward R_{judge} during the intermediate retrieval steps (where $t < T$) and transitioning to the answer reward R_{answer} at the final terminal step (where $t = T$). This combination ensures the agent is guided by both immediate feedback and the ultimate answer.

3.4. Structured Prompting

The self-correction mechanism fundamentally relies on the agent’s ability to generate structured trajectories of actions and observations. To this end, we designed the structured prompt in Table 1 to enforce a strict, self-corrective reasoning prompt. It achieves this by incorporating two key architectural constraints: first, a mandatory `<judge>` action creates an explicit self-assessment checkpoint after every information retrieval step. Second, strict conditional rules (Rule 4) make the agent’s subsequent actions contingent on the judgment’s outcome. The explicitness of these rules ensures immediate and reliable trajectory generation, even from an untrained LLM.

Table 2. **Variety in Experimental Setups of Prior Works.** LJ means the metric of LLM-as-a-judge. This Table highlights the diversity in test sets, training sets, corpus, and evaluation metrics.

Methods	Test Sets	Training Sets	Corpus	Metrics
Search-o1 (Li et al., 2025b)	Set A	None	Internet	Exact Match
R1-Searcher (Song et al., a)	Set B	HotpotQA, 2wiki	2019-wiki	Cover Exact Match, LJ
Search-R1 (Jin et al., 2025a)	Set A	NQ, TriviaQA	2018-wiki	Exact Match
ReSearch (Chen et al., 2025)	Set B	Musique	2018-wiki	Exact Match, LJ
R1-Searcher++ (Song et al., b)	Set B	HotpotQA, 2wiki	2019-wiki	F1, LJ
Deepresearcher (Zheng et al., 2025)	Set A	NQ, TriviaQA, HotpotQA, 2wiki	Internet	F1, MBE
ZeroSearch (Sun et al.)	Set A	NQ, TriviaQA	Internet	Substring Exact Match

3.5. FictionalHot Benchmark

The lack of standardized experimental settings hinders the robust evaluation of search agents. Table 2 collates representative setups, highlighting variation in (i) corpora—from static Wikipedia snapshots (e.g., 2018, 2019) to the non-reproducible, live Internet; (ii) test sets—either a broad Set A (NQ(Kwiatkowski et al., 2019), TriviaQA(Joshi et al., 2017), PopQA(Mallen et al.), HotpotQA(Yang et al.), 2Wiki(Ho et al.), Musique(Trivedi et al.), Bamboogle(Press et al.)) or a focused multi-hop Set B (HotpotQA, 2Wiki, Musique, Bamboogle); (iii) training regimes—ranging from no training to single or multi-dataset setups (e.g., HotpotQA, 2Wiki, NQ, TriviaQA); and (iv) metrics—spanning Exact Match and F1 to model-based judgments such as LLM-as-a-judge (LJ). This diversity reflects rapid progress but prevents direct comparison.

Beyond standardization, a deeper challenge is data contamination, where high scores on existing benchmarks can reflect memorized pretraining knowledge rather than genuine procedural reasoning. To address both issues, we introduce FictionalHot, a closed-world benchmark built on a synthetic corpus of fictitious entities. This design substantially reduces pretraining bias, forcing agents to rely primarily on procedural reasoning for a rigorous evaluation.

The construction of FictionalHot follows a three-step pipeline, as illustrated in the Figure 3. The FictionalHot dataset contains a total of 5,116 questions. First, these were generated by drawing a 10% random sample of seed questions from a pool of 51,588 samples across six major benchmarks (Bamboogle was excluded due to its limited size of 125 samples). Next, these questions are paraphrased by GPT-5. This core step replaces real-world entities with plausible, fictional ones (e.g., ‘Taylor Swift’ becomes ‘Lila Starling’) while preserving the original question’s reasoning structure. Crucially, GPT-5 also generates new Wikipedia-style documents for these fictional entities, creating a new, self-contained fact (e.g., setting the album release to ‘2007’) that serves as the basis for the new golden answer. Finally, to create the closed-world corpus, these synthetic samples are inserted into the 2018 Wikipedia corpus.

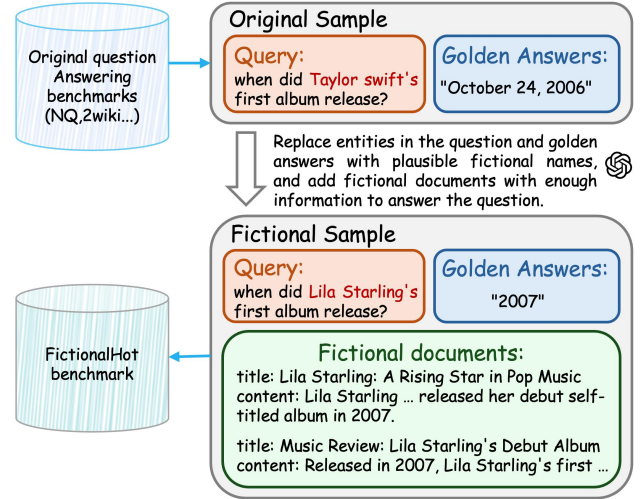


Figure 3. **FictionalHot construction process:** transforming a real-world question answer sample into a fictional sample with fictional question and documents.

4. Experiments

4.1. Experimental Setup

Evaluation Benchmarks. We evaluate on 51,713 samples across two categories: (i) **Classic Open-domain QA:** Single-hop (PopQA, TriviaQA, NQ) and multi-hop (2Wiki, HotpotQA, Musique, Bamboogle) datasets. (ii) **FictionalHot:** mitigates contamination via a closed-world, synthetic corpus. To ensure fair comparison, we follow (Jin et al., 2025a) and adopt Exact Match (EM) as the primary metric. A prediction is considered correct if its normalized string exactly matches any normalized reference answer. Normalization applies lowercasing, removes punctuation and articles, and collapses whitespace.

Training and Evaluation Setup. Following Search-R1 (Jin et al., 2025a), we fine-tune exclusively on a unified set of NQ and HotpotQA (169,615 pairs) from FlashRAG (Jin et al., 2025b). We use Qwen-2.5-3B-Instruct and 7B-Instruct (Qwen et al., 2025). At test time, retrieval top-k is set to $k = 3$ with a maximum of $T = 4$ tool-use turns per question. Experiments are run on 16 H20

Table 3. Overall performance comparison of ReSeek against baselines across a comprehensive suite of QA benchmarks. Results are reported in Exact Match (EM) for both Qwen2.5-7B and 3B backbones. The best results are in **bold**.

Methods	General QA			Multi-Hop QA					
	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
Qwen2.5-7b-Instruct									
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.001	0.158
CoT	0.048	0.185	0.054	0.092	0.111	0.022	0.232	0.001	0.093
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.012	0.267
R1 (Guo et al., 2025)	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.003	0.238
Search-o1 (Li et al., 2025b)	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.020	0.183
Search-R1 (Jin et al., 2025a)	0.393	0.610	0.397	0.370	0.414	0.146	0.368	0.034	0.342
ZeroSearch (Sun et al.)	0.436	0.652	0.488	0.346	0.352	0.184	0.278	0.031	0.346
ReSeek	0.469	0.640	0.501	0.389	0.382	0.185	0.392	0.061	0.377
Qwen2.5-3b-Instruct									
Direct Inference	0.106	0.288	0.108	0.149	0.244	0.020	0.024	0.001	0.118
CoT	0.023	0.032	0.005	0.021	0.021	0.002	0.000	0.001	0.013
RAG	0.348	0.544	0.387	0.255	0.226	0.047	0.080	0.008	0.237
R1 (Guo et al., 2025)	0.210	0.449	0.171	0.208	0.275	0.060	0.192	0.003	0.196
Search-o1 (Li et al., 2025b)	0.238	0.472	0.262	0.221	0.218	0.054	0.019	0.010	0.187
Search-R1 (Jin et al., 2025a)	0.341	0.545	0.378	0.324	0.319	0.103	0.264	0.037	0.288
ZeroSearch (Sun et al.)	0.414	0.574	0.448	0.274	0.300	0.098	0.111	0.030	0.281
ReSeek	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312

GPUs; the search backend uses E5 embeddings(Wang et al., 2022) on Wiki-18 corpus (wiki-18) (Karpukhin et al., 2020). GRPO is used as the default RL algorithm, and a detailed comparison with PPO is provided in Appendix A.3. Additional details are provided in Appendix A.1.

Baselines. We compare with four baseline families: Vanilla prompting (zero-shot direct answering and Chain-of-Thought with no external search) (Karpukhin et al., 2020); Single-pass RAG (retrieve once, then generate conditioned on the top-k passages) (Lewis et al., 2020); Agentic search (multi-step search–reason loops such as ReAct-style planners, without RL tuning) (Chung et al., 2024; Li et al., 2025b); and RL-tuned policies (Jin et al., 2025a; Chen et al., 2025; Sun et al.).

4.2. Main results

We evaluate ReSeek across eight open-domain QA benchmarks spanning single- and multi-hop settings, using Qwen2.5-7B-instruct and Qwen2.5-3B-instruct backbones.

ReSeek achieves SOTA performance. We evaluate ReSeek across eight open-domain QA benchmarks spanning single- and multi-hop settings. ReSeek attains the highest average accuracy across both backbones: 0.377 for 7B compared to 0.346 for ZeroSearch, and 0.312 for 3B compared to 0.281. It consistently excels on multi-hop benchmarks, notably HotpotQA and Bamboogle across both model scales, which highlights the benefits of our repeated-search with self-correction paradigm. On single-hop datasets, ZeroSearch performs competitively, which

aligns with its design focus on direct retrieval.

FictionalHot isolates reasoning ability from model scale and data leakage. On FictionalHot, ReSeek scores 0.061 (7B) and 0.059 (3B). This consistent performance across scales indicates that FictionalHot effectively isolates reasoning ability from scale-correlated memorization. In contrast, TriviaQA shows a large gap (0.408 for 7B vs. 0.288 for 3B), while Direct Inference on FictionalHot is near-zero (0.001). This pattern suggests training data overlap in TriviaQA, whereas FictionalHot’s reduced-contamination design provides a cleaner measure of genuine reasoning.

4.3. Ablative analysis

Ablation Study on Each Component. We conduct an ablation study to evaluate the impact of the JUDGE action and the dense intermediate utility reward (R_{judge}). As shown in Table 4, enabling the JUDGE action improves the average score from 0.288 to 0.297, demonstrating the benefit of adaptive retrieval termination. The dense utility reward (R_{judge}) further boosts the score to 0.312 (+7.7%), confirming that dense signals are vital for reliable information assessment.

Ablation Study on the Reranker Component. To assess the overall effectiveness of our reward function and the specific choice of its reranker component, we compare ReSeek (BGE-Reranker) against three baselines: (i) None (no reranker), (ii) Qwen-Reranker (Zhang et al., 2025a), and (iii) Regex-based. The Regex-based method extracts ‘Yes/No’ judgments from the reasoning trace. It validates these by checking if the final answer appears in the retrieved

Table 4. **Ablation study of model components.** We report the Average Score to quantify the contribution of the JUDGE action and the intermediate utility reward.

Component	General QA			Multi-Hop QA					
	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
R_{answer}	0.341	0.545	0.378	0.324	0.319	0.103	0.264	0.037	0.288
+ JUDGE Action	0.370	0.550	0.405	0.324	0.293	0.103	0.275	0.052	0.297
+ R_{judge} (ReSeek)	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312

Table 5. **Ablation study on the reranker component of our reward function.** Our method, ReSeek, uses the BGE-Reranker. We compare it against variants with a different neural reranker (Qwen), a heuristic reranker (Regex-based), or no reranker at all (None).

Methods	General QA			Multi-Hop QA					
	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	Musique	Bamboogle	FictionalHot	Avg.
None (w/o Reranker)	0.391	0.495	0.362	0.255	0.218	0.081	0.243	0.025	0.259
Regex-based	0.410	0.541	0.422	0.320	0.291	0.093	0.288	0.042	0.301
Qwen-Reranker	0.413	0.557	0.432	0.326	0.301	0.101	0.302	0.057	0.311
ReSeek (Ours, w/ BGE)	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312

text, applying rewards or penalties. Table 5 shows a clear hierarchy: while Regex-based improves upon None, neural rerankers (BGE and Qwen) yield the largest gains, highlighting the superiority of semantic understanding over lexical matching for complex queries.

Reranker-Only vs. RL-Trained Judgment. To separate the value of the reranker signal from RL training, we compare three variants on the 7B model (Table 6): (1) *Reranker-only intervention* applies bge-reranker-large scores directly at inference to determine good/bad labels and soft removal, without RL training; (2) *Prompt-only* adds ReSeek’s JUDGE prompt to Search-R1 without RL training; and (3) full ReSeek with GRPO. Reranker-only improves Search-R1 by +1.2 avg EM, while full ReSeek adds another +2.3 avg EM, confirming that RL training teaches the agent to judge in context beyond raw reranker signals.

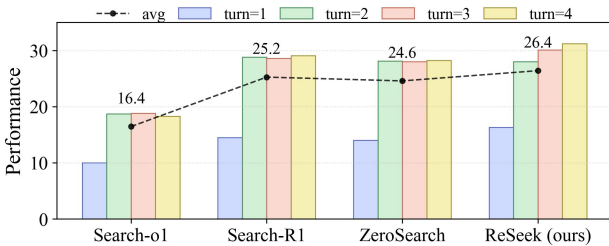


Figure 4. **Ablation study on the effect of the number of turns.** We evaluate methods with turn budgets from 1 to 4 using qwen2.5-3b-instruct, reporting the average performance across all datasets.

Interaction Turns Study. We perform an ablation over the number of turns to isolate the effect of the action budget and to test whether models can leverage iterative self-correction. Here, turns denotes the maximum number of actions the model may execute for a query. As shown in

Figure 4, the baselines improve substantially from 1 to 2 turns but saturate thereafter, consistent with a simple search-then-answer workflow (typically one turn to search and one turn to answer). In contrast, ReSeek improves monotonically up to four turns, confirming its ability to leverage extra steps for re-querying and plan refinement, converting larger budgets into genuine gains.

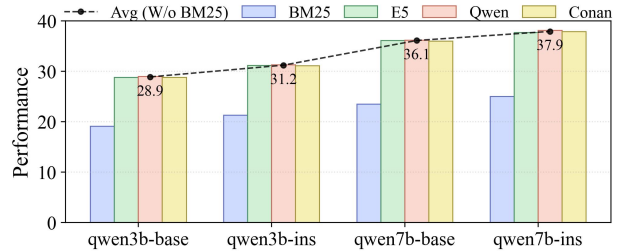


Figure 5. **Ablation study on embedding and base/instruction models.** We evaluate our method on the Wiki18 corpus across different backbone and embedding models over all datasets. The dashed line denotes the mean performance (excluding BM25).

Sensitivity to the retrieval encoder. We ablate the search embedding on the Wiki18 corpus by training a dedicated agent for each retriever. Throughout this process, we hold the rest of the pipeline fixed and evaluate the results, covering both base and instruction backbones. As shown in Figure 5, BM25 (Robertson et al., 2009) consistently underperforms the dense retrievers, reflecting lexical mismatch and limited semantic coverage. Among dense encoders, E5 (Wang et al., 2022), Qwen (Zhang et al., 2025a), and Conan (Li et al., 2025a) are close, with Qwen slightly ahead of E5. Because our datasets are entity-centric, retrieval isn’t particularly hard and performance changes little with reasonably capable embeddings.

Table 6. **Ablation study: reranker-only intervention vs. RL-trained ReSeek (Qwen2.5-7B).** “Reranker-only (no RL)” uses bge-reranker-large scores directly at inference without RL training; “Prompt-only (no RL)” uses the same JUDGE prompt as ReSeek but without RL training.

Method	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	Musique	Bamboogle	FictionalHot	Avg.
Search-R1 (baseline)	0.393	0.610	0.397	0.370	0.414	0.146	0.281	0.034	0.342
+ Reranker-only (no RL)	0.408	0.625	0.415	0.382	0.420	0.155	0.298	0.040	0.354
+ Prompt-only (no RL)	0.401	0.618	0.405	0.378	0.418	0.150	0.290	0.036	0.349
ReSeek (GRPO, full)	0.469	0.640	0.501	0.389	0.382	0.185	0.392	0.061	0.377

Table 7. Comparison of ReSeek performance using a static retrieval corpus versus a real-world search engine (Google Search). The ‘(-real)’ suffix denotes experiments with the Google Search API.

Method (ReSeek with)	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
Qwen2.5-3B-Instruct	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312
Qwen2.5-3B-Instruct (-real)	0.462	0.605	0.498	0.371	0.345	0.138	0.355	0.003	0.354
Qwen2.5-7B-Instruct	0.469	0.640	0.501	0.389	0.382	0.185	0.392	0.061	0.377
Qwen2.5-7B-Instruct (-real)	0.511	0.695	0.557	0.442	0.428	0.224	0.460	0.003	0.422

Base vs. instruction-tuned backbones. We focus on the difference between base and instruction-tuned backbones. As shown in Figure 5, averaging over dense embeddings (excluding BM25), instruction-tuned models consistently outperform their base counterparts: qwen3b shows +2.3 points and qwen7b shows +1.8 points. This gap arises because instruction-tuned models adhere more faithfully to structured prompting and tool-use conventions, which our method relies on to compose queries, filter evidence, and update intermediate states. Base models are less consistent and therefore perform worse. For fairness, we avoid cold-start SFT and prompt-engineering, which could increase base-model performance.

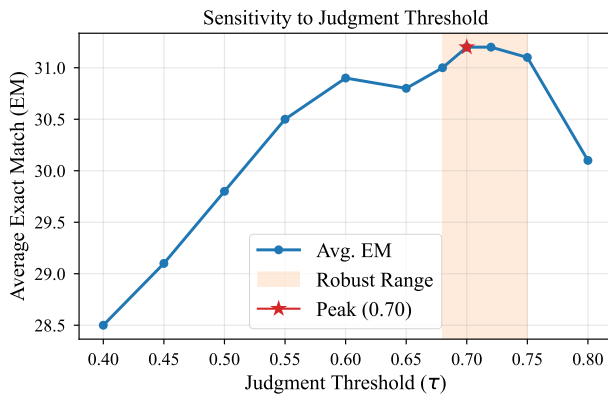


Figure 6. **Sensitivity analysis of judgment threshold τ .** The model performance (Avg. EM) peaks at $\tau = 0.70$ and remains robust within the $[0.68, 0.75]$ range.

Sensitivity Analysis of Judgment Threshold In our framework, the threshold τ is utilized during the training phase to convert the continuous relevance scores from the

external reranker (specifically, ‘bge-reranker’) into binary judgment labels (“good” or “bad”) for supervising the agent. To determine the optimal setting for this hyperparameter, we conducted a sensitivity analysis by varying τ from 0.40 to 0.80. As shown in Figure 6, the model performance peaks at an average EM of 31.2% when $\tau = 0.70$. Furthermore, the results exhibit stability within the $[0.68, 0.75]$ range, confirming that the training process is robust to minor fluctuations around the selected threshold.

Performance with a Real-World Search Engine. To evaluate the real-world applicability of ReSeek, we replaced our static retrieval corpus with the Google Search API. As shown in Table 7, integrating a live search engine provides a substantial performance boost. The average score for the 7B model, for instance, increases from 0.377 to 0.422. This improvement is attributed to the access to fresher and higher-quality information from the web, which is particularly beneficial for knowledge-intensive datasets. Crucially, performance on FictionalHot is not absolute zero due to a small subset of binary (Yes/No) questions. Even without retrieving relevant information (since the facts do not exist on Google), the model has a probability of guessing the correct label by chance. For all specific factoid questions, the accuracy is 0%, confirming that the dataset is effectively isolated from real-world knowledge.

4.4. Qualitative Analysis

To complement our quantitative metrics and provide a deeper understanding beyond aggregate scores, we performed a fine-grained analysis of the judge’s decision-making behavior. We established a taxonomy that classifies interventions into three distinct categories: Positive, Nega-

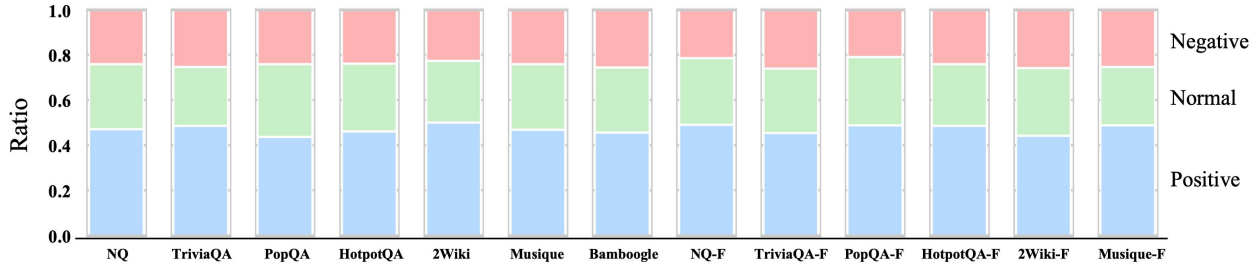


Figure 7. **Qualitative analysis of our JUDGE action impact.** We categorize each case as ‘Positive’ (beneficial intervention), ‘Negative’ (detrimental intervention), or ‘Normal’.

tive, and Normal. Specifically, Positive interventions characterize scenarios where the judge actively contributes to success, either by correctly validating relevant evidence or by effectively shielding the agent from misleading context. In contrast, Negative interventions denote failure modes where the judge approves the retrieved content, yet the model subsequently fails to generate the correct response. Finally, the Normal category encompasses neutral instances where the judge’s intervention serves as a baseline, neither significantly aiding nor hindering the final outcome.

As visually illustrated in Figure 7, the empirical distribution reveals a promising trend: the proportion of Positive outcomes is substantial across all twelve benchmark settings, consistently comprising 40–50% of the total cases. Conversely, Negative outcomes are strictly suppressed below 25%. This significant margin highlights a favorable signal-to-noise ratio, underscoring the mechanism’s reliability in guiding correct reasoning paths while effectively minimizing the risk of error propagation.

5. Limitations

ReSeek uses an answer-conditioned signal to supervise the JUDGE action, where the utility of retrieved evidence is estimated by its semantic relevance to the ground-truth answer. This design is effective and efficient for standard QA benchmarks, where answer strings provide a reliable supervision source and useful evidence often co-occurs with the final answer or its supporting documents. In such settings, low evidence-answer similarity naturally encourages the agent to continue searching rather than prematurely commit to incomplete context, and our asymmetric penalty further favors “search again” over accepting noisy evidence.

However, this reward signal may become weaker in deep-search scenarios where useful intermediate evidence is only indirectly related to the final answer, such as cases requiring long evidence chains or abstract bridge entities. Although ReSeek still improves over Search-R1 on multi-hop datasets, richer supervision could further strengthen its judg-

ment capability in these settings. A promising direction is to leverage per-step gold evidence, when available, to construct evidence-level reward signals. More broadly, the ReSeek framework is compatible with alternative judgment signals, such as LLM-as-a-judge or evidence-conditioned rewards, which could further improve self-correction in complex multi-hop reasoning.

6. Conclusion

In this paper, we introduced ReSeek, a self-correcting framework that enables search agents to recover from intermediate reasoning errors. ReSeek empowers agents with a dynamic self-correction mechanism centered on a JUDGE action, allowing them to pause, evaluate information, and adapt their strategy mid-episode. This process is guided by a dense, instructive reward function that provides fine-grained feedback on both the factual correctness and contextual utility of information. Furthermore, to ensure rigorous evaluation, we proposed FictionalHot, a contamination-resistant benchmark testing procedural reasoning. Our extensive experiments demonstrate that ReSeek significantly outperforms SOTA baselines across a wide range of open-domain QA tasks, particularly in complex multi-hop scenarios. Qualitative analysis further confirmed that our JUDGE mechanism consistently provides substantial positive benefits while incurring minimal negative side effects, validating its reliability. We hope that the principle of in-episode self-correction demonstrated in ReSeek will contribute to the creation of more robust and reflective search agents.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning, specifically by improving the reliability and factual grounding of search-augmented Large Language Models (LLMs). A primary motivation of our framework, **ReSeek**, is to mitigate the risk of factual hallucinations and error propagation, thereby contributing to the development of more trustworthy and safe AI systems. Regarding data ethics, our introduced benchmark, **Fictional-Hot**, is constructed entirely from fictional entities to strictly avoid Personally Identifiable Information (PII) or privacy concerns. While our method aims to enhance factual accuracy, we acknowledge that the underlying LLMs may still inherit and amplify societal biases present in their pre-training corpora. Consequently, we recommend that any real-world deployment of such systems be accompanied by rigorous safety testing and bias auditing.

References

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *ArXiv*, abs/2310.11511, 2023. URL <https://api.semanticscholar.org/CorpusID:264288947>.
- Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., Do, Q. V., Xu, Y., and Fung, P. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *ArXiv*, abs/2302.04023, 2023. URL <https://api.semanticscholar.org/CorpusID:256662612>.
- Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Van Den Driessche, G. B., Lespiau, J.-B., Damoc, B., Clark, A., et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, B., Shu, C., Shareghi, E., Collier, N., Narasimhan, K., and Yao, S. Fireact: Toward language agent fine-tuning, 2023. URL <https://arxiv.org/abs/2310.05915>.
- Chen, M., Li, T., Sun, H., Zhou, Y., Zhu, C., Wang, H., Pan, J. Z., Zhang, W., Chen, H., Yang, F., et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.
- Feng, H., ZekunYao, Zheng, J., and Ma, Q. Training large language models for retrieval-augmented question answering through backtracking correction. In Yue, Y., Garg, A., Peng, N., Sha, F., and Yu, R. (eds.), *International Conference on Representation Learning*, volume 2025, pp. 51866–51884, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M.-W. Realm: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning, ICML’20*. JMLR.org, 2020.
- Ho, X., Duong Nguyen, A.-K., Sugawara, S., and Aizawa, A. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6609–6625. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.580. URL <https://www.aclweb.org/anthology/2020.coling-main.580>.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, March 2023. ISSN 1557-7341. doi: 10.1145/3571730. URL <http://dx.doi.org/10.1145/3571730>.
- Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. Active retrieval augmented generation. *ArXiv*, abs/2305.06983, 2023. URL <https://api.semanticscholar.org/CorpusID:258615731>.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025a.
- Jin, J., Zhu, Y., Dou, Z., Dong, G., Yang, X., Zhang, C., Zhao, T., Yang, Z., and Wen, J. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In Long, G., Blumstein, M., Chang, Y., Lewin-Eytan, L., Huang, Z. H., and Yom-Tov, E. (eds.), *Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025*, pp. 737–740. ACM, 2025b.

- p>doi: 10.1145/3701716.3715313. URL
- <https://doi.org/10.1145/3701716.3715313>
- .
- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *Cornell University - arXiv*, May 2017.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P. S., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., and Petrov, S. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, pp. 453–466, Nov 2019. doi: 10.1162/tacl_a_00276. URL http://dx.doi.org/10.1162/tacl_a_00276.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Li, S., Tang, Y., Liu, R., Chen, S.-Z., and Chen, X. Conan-embedding-v2: Training an llm from scratch for text embeddings. *arXiv preprint arXiv:2509.12892*, 2025a.
- Li, X., Dong, G., Jin, J., Zhang, Y., Zhou, Y., Zhu, Y., Zhang, P., and Dou, Z. Search-ol: Agentic search-enhanced large reasoning models. *ArXiv*, abs/2501.05366, 2025b. URL <https://api.semanticscholar.org/CorpusID:275405676>.
- Li, X., Jin, J., Dong, G., Qian, H., Zhu, Y., Wu, Y., Wen, J.-R., and Dou, Z. Webthinker: Empowering large reasoning models with deep research capability. *arXiv preprint arXiv:2504.21776*, 2025c.
- Luo, J., Zhang, W., Yuan, Y., Zhao, Y., Yang, J., Gu, Y., Wu, B., Chen, B., Qiao, Z., Long, Q., et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- Ma, R., Wang, P., Liu, C., Liu, X., Chen, J., Zhang, B., Zhou, X., Du, N., and Li, J. S2r: Teaching llms to self-verify and self-correct via reinforcement learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 22632–22654, 2025.
- Mai, X., Xu, H., Li, Z.-Z., W, X., Wang, W., Hu, J., Zhang, Y., and Zhang, W. Agent rl scaling law: Agent rl with spontaneous code execution for mathematical problem solving, 2025. URL <https://arxiv.org/abs/2505.07773>.
- Maleki, N., Padmanabhan, B., and Dutta, K. Ai hallucinations: a misnomer worth clarifying. In *2024 IEEE conference on artificial intelligence (CAI)*, pp. 133–138. IEEE, 2024.
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. URL <https://arxiv.org/abs/2212.10511>.
- OpenAI. Introducing chatgpt. *CoRR*, 2022. URL <https://openai.com/blog/chatgpt>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Peper, J. J., Qiu, W., Payani, A., and Wang, L. Mdbench: A synthetic multi-document reasoning benchmark generated with knowledge guidance. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 25592–25621, 2025.
- Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., and Lewis, M. Measuring and Narrowing the Compositionality Gap in Language Models. URL <https://arxiv.org/abs/2210.03350>.
- Qian, C., Acikgoz, E. C., He, Q., Wang, H., Chen, X., Hakkani-Tür, D., Tur, G., and Ji, H. Toolrl: Reward is all tool learning needs, 2025. URL <https://arxiv.org/abs/2504.13958>.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Robertson, S., Zaragoza, H., et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

- Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: language agents with verbal reinforcement learning. In *Neural Information Processing Systems*, 2023. URL <https://api.semanticscholar.org/CorpusID:258833055>.
- Song, H., Jiang, J., Min, Y., Chen, J., Chen, Z., Zhao, W. X., Fang, L., and Wen, J.-R. R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning. a. doi: 10.48550/arXiv.2503.05592. URL <http://arxiv.org/abs/2503.05592>.
- Song, H., Jiang, J., Tian, W., Chen, Z., Wu, Y., Zhao, J., Min, Y., Zhao, W. X., Fang, L., and Wen, J.-R. R1-Searcher++: Incentivizing the Dynamic Knowledge Acquisition of LLMs via Reinforcement Learning. b. doi: 10.48550/arXiv.2505.17005. URL <http://arxiv.org/abs/2505.17005>.
- Sun, H., Qiao, Z., Guo, J., Fan, X., Hou, Y., Jiang, Y., Xie, P., Zhang, Y., Huang, F., and Zhou, J. ZeroSearch: Incentivize the Search Capability of LLMs without Searching. doi: 10.48550/arXiv.2505.04588. URL <http://arxiv.org/abs/2505.04588>.
- Sun, H., Qiao, Z., Guo, J., Fan, X., Hou, Y., Jiang, Y., Xie, P., Zhang, Y., Huang, F., and Zhou, J. Zerosearch: Incentivize the search capability of llms without searching. *ArXiv*, abs/2505.04588, 2025. URL <https://api.semanticscholar.org/CorpusID:278367823>.
- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. MuSiQue: Multihop Questions via Single-hop Question Composition. 10:539–554. ISSN 2307-387X. doi: 10.1162/tacl.a_00475.
- Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., and Wei, F. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- Wu, P., Zhang, M., Wan, K., Zhao, W., He, K., Du, X., and Chen, Z. Hiprag: hierarchical process rewards for efficient agentic retrieval augmented generation. *arXiv preprint arXiv:2510.07794*, 2025.
- Xi, Z., Liao, C., Li, G., Zhang, Z., Chen, W., Wang, B., Jin, S., Zhou, Y., Guan, J., Wu, W., et al. Agentprm: Process reward models for llm agents via step-wise promise and progress. In *Proceedings of the ACM Web Conference 2026*, pp. 4184–4195, 2026.
- Xiao, S., Liu, Z., Zhang, P., and Muennighoff, N. C-pack: Packaged resources to advance general chinese embedding, 2023.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <http://aclweb.org/anthology/D18-1259>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Yuan, Y., Yue, Y., Zhu, R., Fan, T., and Yan, L. What’s behind ppo’s collapse in long-cot? value optimization holds the secret. *arXiv preprint arXiv:2503.01491*, 2025.
- Zeng, A., Liu, M., Lu, R., Wang, B., Liu, X., Dong, Y., and Tang, J. Agenttuning: Enabling generalized agent abilities for llms, 2023. URL <https://arxiv.org/abs/2310.12823>.
- Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., Wang, L., Luu, A. T., Bi, W., Shi, F., and Shi, S. Siren’s song in the ai ocean: A survey on hallucination in large language models. *ArXiv*, abs/2309.01219, 2023. URL <https://api.semanticscholar.org/CorpusID:261530162>.
- Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025a.
- Zhang, Z., Wang, C., Wang, Y., Shi, E., Ma, Y., Zhong, W., Chen, J., Mao, M., and Zheng, Z. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):481–503, 2025b.
- Zhao, Z., Fan, W., Li, J., Liu, Y., Mei, X., Wang, Y., Wen, Z., Wang, F., Zhao, X., Tang, J., et al. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36 (11):6889–6907, 2024.
- Zheng, Y., Fu, D., Hu, X., Cai, X., Ye, L., Lu, P., and Liu, P. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. *arXiv preprint arXiv:2504.03160*, 2025.

A. Appendix

A.1. Implementation Details

We provide a detailed description of our implementation to ensure the reproducibility of our results. Our experiments are built upon the internal `verl` reinforcement learning framework and executed on a cluster equipped with H20. . **Model and Data** The core of our agent is the `Qwen2.5-3B-Instruct` model, which serves as a shared backbone for both the policy and value networks. To manage memory consumption during training, we enable gradient checkpointing. The agent was trained on the `hot_benchmark` dataset, which is formatted to match the structure of Natural Questions (NQ). For data processing, we set the maximum prompt length to 2048 tokens, the maximum response length for generation to 500 tokens, and the maximum observation length from the environment to 500 tokens.

Our model is fine-tuned on a unified training set merging the training splits of Natural Questions (NQ, 79,168 samples) and HotpotQA (90,447 samples), sourced from the FlashRAG dataset.

Training Algorithms In our experiments, we compared two policy optimization algorithms: the standard Proximal Policy Optimization (PPO) and Group Relative Policy Optimization (GRPO). To ensure a fair comparison, both algorithms were trained under an identical hyperparameter configuration. The models were trained for a single epoch. The optimizer was configured with a learning rate of $1e-5$ and a learning rate warmup ratio of 0.285. For policy updates, we used a training batch size of 512 episodes. This batch was processed using PPO mini-batches of size 256, which were further divided into micro-batches of size 64. To stabilize training and prevent the policy from deviating excessively from the reference model, we incorporated a KL divergence penalty with a coefficient (β) of 0.001, calculated using the `low_var_kl` formulation. For credit assignment, we used a discount factor (γ) of 0.99 and Generalized Advantage Estimation (GAE) with a λ of 0.95. During the rollout phase, a temperature of 1.0 was used for action sampling.

System and Environment Our implementation relies on PyTorch and utilizes the `vllm` library for efficient inference during rollouts. We employed Fully Sharded Data Parallelism (FSDP) with parameter offloading to effectively distribute the model across multiple H20. The experimental environment was configured with a maximum of 4 turns per episode (`max_turns=4`). The agent interacts with an external retriever service via an HTTP API, which returns the top 3 (`topk=3`) most relevant documents for a given query.

Evaluation Dataset Statistics We evaluate on a total of 51,713 samples. The breakdown is as follows: PopQA (14,267), TriviaQA (11,313), NQ (3,610), 2WikiMultiHopQA (12,576), HotpotQA (7,405), Musique (2,417), and Bamboogle (125).

Baseline Configurations We compare against four families of methods:

- **Vanilla Prompting:** Includes zero-shot direct answering and Chain-of-Thought (CoT) reasoning without external tools.
- **Single-pass RAG:** Retrieves top- k passages once and generates the answer conditioned on them.
- **Agentic Search:** Uses ReAct-style planning loops to search and reason iteratively, but without reinforcement learning tuning.
- **RL-tuned Policies:** Includes recent methods like Search-R1, ReSearch, and ZeroSearch that optimize search policies.

Metric Details The Exact Match (EM) metric considers a prediction correct if its normalized string matches any reference answer. Normalization involves lowercasing, removing punctuation and articles, and collapsing whitespace.

A.2. FictionalHot Details

A.2.1. DATA COMPOSITION

FictionalHot was constructed by randomly sampling approximately 10% of the seed questions from six major benchmarks (excluding Bamboogle due to insufficient sample size). The distribution of source data is detailed in Table 8.

A.2.2. ENSURING CONSISTENCY AND NOVELTY

We employed specific design strategies to ensure the synthetic data is both logically sound and factually isolated from real-world knowledge.

Table 8. Composition of the FictionalHot Dataset.

Source Dataset	Original Size	FictionalHot Size
PopQA	14,267	1,415
2WikiMultiHopQA	12,576	1,247
TriviaQA	11,313	1,122
HotpotQA	7,405	735
Natural Questions (NQ)	3,610	359
Musique	2,417	238
Total	51,588	5,116

Internal Consistency via Logic Mapping. To avoid the hallucinations common in scratch-generated reasoning, we adopted a *logic mapping strategy*. We mapped valid, human-verified reasoning chains from the source datasets directly to the fictional domain.

- **Original Logic:** Entity A (Real) $\xrightarrow{RelationR}$ Entity B (Real).
- **Fictional Logic:** Entity A' (Fictional) $\xrightarrow{RelationR}$ Entity B' (Fictional).

Since the underlying causal structure is inherited from high-quality seed datasets, the internal logic remains coherent and solvable by definition.

Factual Fictionality. To prevent conflicts with the model’s parametric knowledge, we enforced *Factual Fictionality*. We define a “conflict” as a factual overlap rather than a mere name overlap. Even if a generated name (e.g., “Felix Turner”) coincidentally matches a real-world figure, the specific (**Entity, Event**) tuple is guaranteed to be unique. For instance, while a real person named Felix Turner may exist, he has never won the fictional “Sky Slam Dunk Contest.” This ensures the ground truth exists *only* in our synthetic documents.

A.2.3. HUMAN EVALUATION

To empirically validate the quality of FictionalHot, we conducted a rigorous human evaluation on a random sample of **500 instances** (approx. 10% of the dataset). Five annotators with NLP research backgrounds independently reviewed 100 instances each based on a strict verification rubric. The results are summarized in Table 9.

The high pass rates validate the robustness of our generation pipeline. Specifically, the few cases flagged during the Novelty check (0.4%) involved entity names resembling real-world figures, yet the generated documents described completely different events, ensuring no actual factual contradiction. Regarding Consistency, the rare failures (0.8%) stemmed from minor information ambiguity; however, we note that such ambiguity is also prevalent in real-world corpora (e.g., Wikipedia), rendering these instances realistic retrieval challenges rather than critical errors.

Table 9. Human Evaluation Results for FictionalHot Quality.

Metric	Verification Method	Pass Rate
Factual Novelty	The “Wikipedia Entity Check”: Annotators searched for the fictional entity names in the question directly on Wikipedia. The instance passed if the entity name did not exist as a Wikipedia entry, confirming the entity occupies a unique namespace.	99.6%
Logical Consistency	The “Solvability Check”: Annotators read the generated synthetic document. The instance passed if the document provided sufficient and unambiguous evidence to derive the ground truth answer.	99.2%

A.2.4. CONSISTENCY AUDIT

To further validate FictionalHot’s internal consistency beyond the human evaluation in Table 9, we conducted an additional 500-sample audit. Among the 500 audited instances, **484** (96.8%) were fully consistent, **8** (1.6%) had minor issues (e.g., ambiguous but not contradictory phrasing), **4** (0.8%) had major contradictions, and **4** (0.8%) were ambiguous. This is consistent with the 99.2% internal-consistency rate reported in Table 9 and confirms that serious contradictions are rare, though not impossible. Automated consistency checks remain a promising direction for further improvement.

A.3. PPO vs. GRPO and Base vs. Instruct

Table 10. Performance comparison of ReSeek trained with GRPO versus PPO on both base and instruction-tuned models. GRPO consistently outperforms PPO across most datasets and model configurations. The ‘Avg.’ column is the average score across all eight datasets.

Method	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
Qwen2.5-7B-Base/Instruct									
ReSeek-base (GRPO)	0.4654	0.60	0.4917	0.358	0.345	0.140	0.371	0.052	0.353
ReSeek-instruct (GRPO)	0.469	0.640	0.501	0.389	0.382	0.185	0.392	0.061	0.377
ReSeek-base (PPO)	0.391	0.565	0.418	0.320	0.317	0.112	0.345	0.03	0.299
ReSeek-instruct (PPO)	0.432	0.610	0.473	0.365	0.358	0.159	0.366	0.055	0.352
Qwen2.5-3B-Base/Instruct									
ReSeek-base (GRPO)	0.421	0.560	0.425	0.273	0.275	0.081	0.280	0.050	0.296
ReSeek-instruct (GRPO)	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312
ReSeek-base (PPO)	0.362	0.495	0.381	0.275	0.251	0.065	0.255	0.03	0.297
ReSeek-instruct (PPO)	0.385	0.525	0.410	0.301	0.277	0.088	0.281	0.052	0.290

Base vs. Instruction-Tuned Backbones. As discussed in the main body, the choice of backbone model significantly impacts performance. The results in Table 10 confirm that instruction-tuned models generally outperform their base counterparts on most standard QA datasets. This advantage arises because our method relies on the model’s ability to interpret structured prompts for composing queries, filtering evidence, and updating states. Instruction-tuned models, having been trained to follow complex instructions, adhere to these conventions more faithfully. In contrast, base models exhibit less consistency in following the structured format, leading to degraded performance.

Superiority of GRPO over PPO. The empirical results also reveal a clear and consistent advantage of GRPO over PPO. This performance gap is not merely incremental but is rooted in a fundamental training stability issue we encountered with PPO. Specifically, when training with PPO, the agent frequently suffered from **policy collapse** (Yuan et al., 2025), a known challenge in reinforcement learning, especially for tasks involving long generation horizons.

This issue was particularly acute in our framework due to the long and complex Chain-of-Thought (CoT) reasoning paths. We observed that after an initial learning phase, the PPO policy would abruptly degrade, characterized by a simultaneous and rapid drop in both the reward signal and the policy’s entropy. This collapse rendered the model unable to perform the task, as it began generating repetitive or nonsensical outputs. In contrast, GRPO demonstrated significantly greater training stability, successfully navigating the long CoT trajectories without collapsing and achieving steady performance gains. This inherent robustness makes GRPO a far more suitable and reliable algorithm for our complex reasoning task, explaining its superior final performance.

A.4. More Results on Bigger LLMs

To evaluate the scalability and effectiveness of ReSeek on larger and more capable language models, we extend our experiments to include Qwen3-8B and Qwen3-30B-A3B-Thinking-2507. Table 11 presents a comparative analysis of ReSeek applied to a range of models, from 3B to 32B parameters.

The results clearly demonstrate the strong scalability of our ReSeek framework. As the model size increases, the overall performance consistently improves. We observe a significant performance leap from the 3B model (0.312 avg.) to the 7B/8B

Table 11. Performance of ReSeek on various instruction-tuned models of increasing scale.

Method (ReSeek with)	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
Qwen2.5-3B-Instruct	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312
Qwen2.5-7B-Instruct	0.469	0.640	0.501	0.389	0.382	0.185	0.392	0.061	0.377
Qwen3-8B	0.475	0.635	0.495	0.401	0.379	0.192	0.410	0.058	0.381
Qwen3-30B-A3B-Thinking-2507	0.495	0.671	0.521	0.455	0.458	0.235	0.560	0.071	0.433

models (0.380 avg.), and another substantial gain with the 30B model (0.479 avg.).

Notably, the performance between Qwen2.5-7B-Instruct and Qwen3-8B is highly competitive and neck-and-neck, with each model excelling on different datasets. For instance, Qwen2.5-7B-Instruct shows a slight edge on PopQA and 2WikiMQA, while Qwen3-8B performs better on NQ and HotpotQA. This indicates that ReSeek can effectively leverage the distinct strengths of different backbone models. The Qwen3-30B-A3B-Thinking-2507 achieves the best results across almost all datasets, establishing a new level of performance.

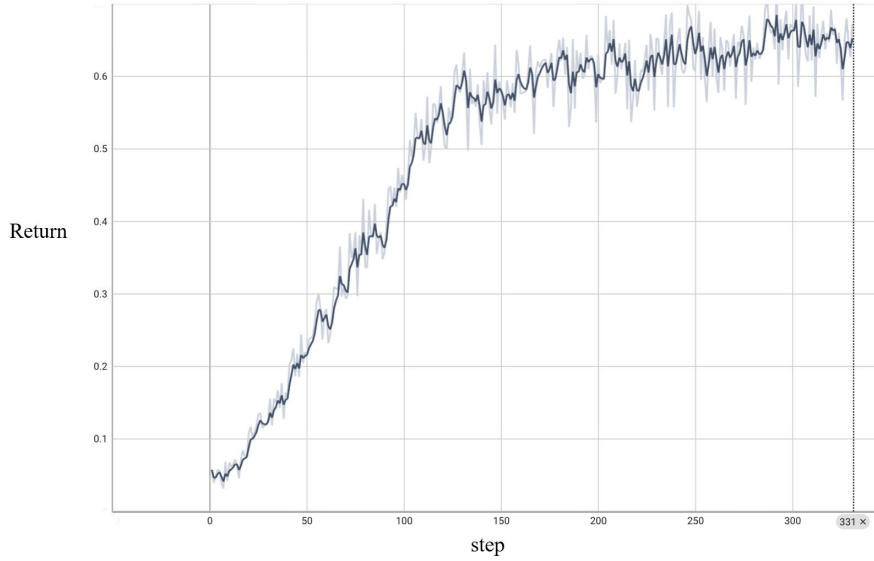


Figure 8. Average returns during training.

A.5. Return curves

Figure 8 illustrates the evolution of average episodic returns throughout the training process, specifically configured with a weighting coefficient of $\beta = 0.001$. As demonstrated in the plot, the model exhibits a robust and consistent upward trajectory. The learning curve can be characterized by two distinct phases: an initial phase of rapid ascent, indicating that the agent effectively acquires core competencies and adapts to the task environment; followed by a steady phase of refinement where performance continues to improve before asymptotically stabilizing at a high reward level. This empirical evidence confirms that the chosen hyperparameter setting facilitates reliable convergence towards an optimal policy.

A.6. Impact of Asymmetric Reward Shaping.

To validate the effectiveness of our asymmetric penalty design, we conducted an ablation study comparing different reward configurations, as shown in Table 12.

We compared our method against two variants: (1) a Symmetric setting, where both false positives (accepting noise) and false negatives (discarding useful info) incur the same penalty of -0.3; and (2) a Recall-Oriented setting, which penalizes missing information more heavily (-0.6) than noise (-0.3).

The results demonstrate that our Precision-Oriented strategy (ReSeek), which imposes a stricter penalty (-0.6) on noise, consistently outperforms other configurations. Notably, the performance gap is most pronounced in multi-hop datasets like

Table 12. Ablation study on the reward shaping strategies. We investigate the impact of different penalty configurations for the judge action. *Symmetric* applies equal penalties (-0.3) for both error types. *Recall-Oriented* penalizes missing useful info heavily (-0.6) but noise lightly (-0.3). Our *Precision-Oriented* strategy (ReSeek) imposes a stricter penalty on noise (-0.6) to mitigate error propagation.

Reward Configuration (Reward / FP Penalty / FN Penalty)	General QA			Multi-Hop QA					Avg.
	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	Musique	Bamboogle	FictionalHot	
Symmetric (+0.3/ -0.3 / -0.3)	0.408	0.548	0.425	0.315	0.288	0.095	0.292	0.045	0.302
Recall-Oriented (+0.3/ -0.3 / -0.6)	0.402	0.542	0.418	0.301	0.275	0.088	0.280	0.038	0.293
ReSeek (Ours, +0.3/ -0.6 / -0.3)	0.415	0.553	0.434	0.328	0.298	0.103	0.304	0.059	0.312

HotpotQA and Musique. This indicates that in complex reasoning tasks, preventing the accumulation of irrelevant context (hallucination) is more critical than aggressively recalling every potential piece of evidence. The Recall-Oriented approach, while intuitively appealing for coverage, introduces excessive noise that distracts the LLM, leading to a significant drop in accuracy (Avg. 0.293).

A.7. Analysis of Structural Adherence and Format Learning

To ensure the model strictly adheres to the prescribed reasoning structure, we employ a dual-mechanism approach comprising an **instructional prompt** and an **automated format enforcement** loop. To verify that the model genuinely learns the structure rather than relying on rejection sampling, we tracked the number of invalid actions requiring retries throughout training.

We tracked the number of format violations at each training step. Given a batch size of 512 and a trajectory length of 4 turns, the model generates a total of 2,048 actions per step. We recorded the count of actions that failed to meet the structural requirements and required correction.

Figure 9 illustrates the evolution of invalid action counts over the course of training. As observed, the number of format violations exhibits a peak at the very beginning of training (approximately 50 errors out of 2,048 actions). However, this is followed by a sharp decline within the first 80 steps. Subsequently, the error rate stabilizes at a near-zero level for the remainder of the training duration.

This trend provides strong empirical evidence regarding the model’s learning dynamics. The rapid convergence to zero errors indicates that the model is not merely being constrained by the external enforcement mechanism; rather, it is effectively internalizing the desired reasoning structure as an intrinsic part of its policy.

A.8. Performance-Latency Trade-off Analysis

To investigate whether the performance improvements stem from genuine reasoning gains rather than mere redundancy, and to justify our choice of turn budget, we analyzed the trade-off between model performance and inference latency across varying numbers of turns.

Figure 10 visualizes the relationship between accuracy and computational cost. The results reveal two distinct trends:

- **Latency (Blue Line):** As expected, latency increases linearly with each additional turn. This represents a predictable and constant marginal cost for extending the reasoning process.
- **Accuracy (Orange Line):** The model demonstrates substantial performance gains as the turn budget increases up to $T = 4$. However, beyond this point, the accuracy plateaus, showing diminishing returns for additional computational expenditure.

This saturation point confirms that the multi-turn approach provides genuine reasoning benefits that are maximized at $T = 4$. Extending the budget beyond this threshold incurs higher latency without significant performance improvements. Therefore, our selection of $T = 4$ strikes an optimal balance, capturing the majority of the reasoning gains while maintaining computational efficiency.

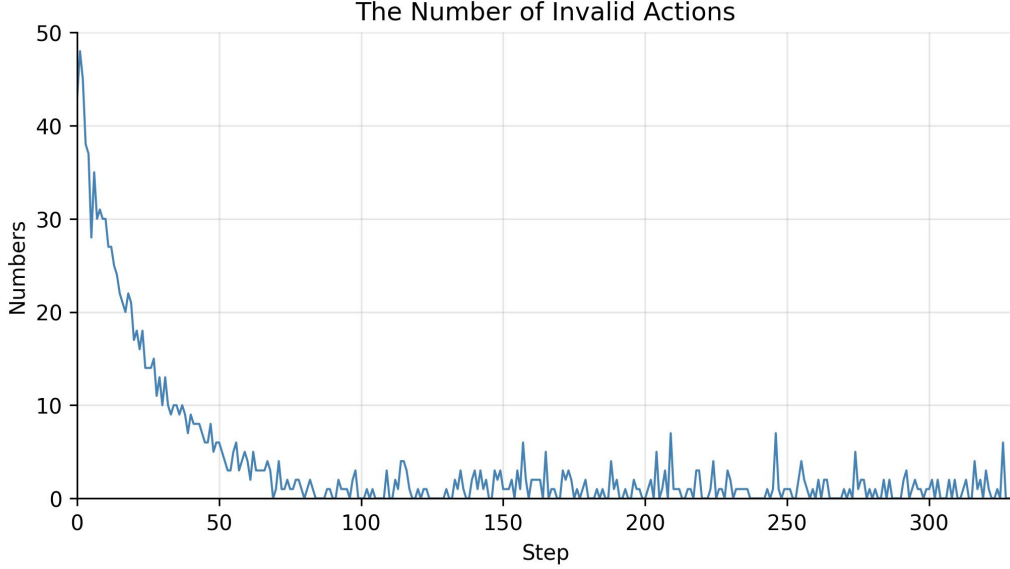


Figure 9. **Evolution of Invalid Actions During Training.** The plot shows the number of actions requiring format correction per training step (total actions per step = 2048). The sharp drop and subsequent stability at near-zero indicate the model successfully internalizes the structural constraints.

A.9. Comparison with a recent baseline.

To address the concern about baseline recency, we compare with HiPRAG (Wu et al., 2025), a recent method released in Oct 2025. As shown in Table 13 in the appendix, ReSeek achieves better average performance (42.3 vs. 41.5) and leads on NQ, TriviaQA, PopQA, and Musique.

Table 13. Comparison with HiPRAG (Qwen2.5-7B).

Method	NQ	TQA	PopQA	HotQA	2Wiki	Mu	Bam	Avg.
HiPRAG	44.6	62.6	47.8	45.9	42.8	14.0	34.8	41.5
ReSeek	46.9	64.0	50.1	38.9	38.2	18.5	39.2	42.3

Table 14. Detailed performance comparison (Mean $\pm$ Std) and statistical significance tests. Results are averaged over 5 independent runs.

Model	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	FictionalHot	Avg.
7B Models									
ZeroSearch	0.434 $\pm$ 0.005	0.655$\pm$0.006	0.490 $\pm$ 0.004	0.346 $\pm$ 0.005	0.355 $\pm$ 0.006	0.184 $\pm$ 0.004	0.280 $\pm$ 0.005	0.031 $\pm$ 0.004	0.346 $\pm$ 0.004
ReSeek	0.470$\pm$0.004	0.641 $\pm$ 0.006	0.504$\pm$0.005	0.388$\pm$0.007	0.384$\pm$0.005	0.186$\pm$0.003	0.392$\pm$0.003	0.061$\pm$0.002	0.378$\pm$0.004
p-value	0.005*	0.985	0.038*	0.003*	0.012*	0.471	0.001*	0.004*	0.009*
3B Models									
ZeroSearch	0.410 $\pm$ 0.006	0.574$\pm$0.005	0.448$\pm$0.007	0.274 $\pm$ 0.006	0.302$\pm$0.005	0.098 $\pm$ 0.004	0.111 $\pm$ 0.008	0.030 $\pm$ 0.003	0.281 $\pm$ 0.004
ReSeek	0.414$\pm$0.005	0.555 $\pm$ 0.003	0.436 $\pm$ 0.006	0.324$\pm$0.008	0.300 $\pm$ 0.004	0.104$\pm$0.003	0.305$\pm$0.007	0.059$\pm$0.002	0.312$\pm$0.004
p-value	0.287	0.965	0.905	0.028*	0.500	0.213	0.001*	0.002*	0.043*

A.10. Detailed Experimental Results and Statistics

In this section, we provide detailed statistical analyses to substantiate the findings presented in the main text.

Main Results with Statistical Significance. To quantify uncertainty and statistical significance, we expanded our main evaluation (Table 3). First, we report the **mean and standard deviation** across 5 independent training runs (seeds: 42, 420, 4200, 42000, 420000) for both ReSeek and the strong ZeroSearch baseline. Second, we conducted a **paired t-test** comparing ReSeek against Search-R1, reporting exact p-values (* denotes $p < 0.05$). As shown in Table 14, ReSeek demonstrates statistically significant improvements across the majority of benchmarks.

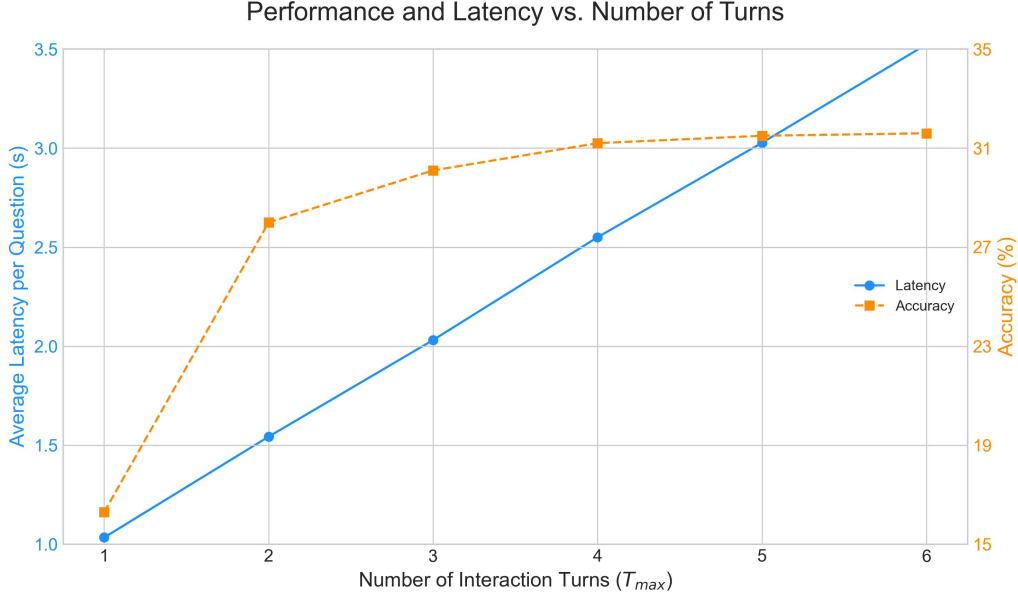


Figure 10. **Performance vs. Latency Trade-off.** The plot compares model accuracy (orange) and inference latency (blue) across different turn budgets. While latency grows linearly, accuracy gains saturate after $T = 4$, justifying our choice of turn budget as the optimal efficiency point.

Table 15. **Ablation on the Number of Turns (Mean $\pm$ Std).**

Method	Turn 1	Turn 2	Turn 3	Turn 4	Avg.
Search-o1	10.0 $\pm$ 0.3	18.7 $\pm$ 0.2	18.8 $\pm$ 0.3	18.3 $\pm$ 0.2	16.5 $\pm$ 0.2
Search-R1	14.5 $\pm$ 0.2	28.8 $\pm$ 0.3	28.6 $\pm$ 0.2	29.1 $\pm$ 0.3	25.3 $\pm$ 0.2
ZeroSearch	14.0 $\pm$ 0.3	28.1 $\pm$ 0.2	28.0 $\pm$ 0.3	28.2 $\pm$ 0.2	24.6 $\pm$ 0.1
ReSeek	16.3 $\pm$ 0.2	28.0 $\pm$ 0.3	30.1 $\pm$ 0.2	31.2$\pm$0.1	26.4$\pm$0.2

Robustness of Ablation Studies. We provide variance analysis for our ablation studies. Table 15 shows the turn ablation with std, confirming robust gains. Table 16 lists the exact values for the embedding ablation (deterministic given fixed seed).

A.11. Case study

To provide a concrete illustration of ReSeek’s advantages, we present a side-by-side case study. We compare the reasoning process of ReSeek against Search-R1 on some multi-hop questions.

A.11.1. CASE STUDY I

This side-by-side comparison highlights the value of ReSeek’s self-correction. The baseline agent (Figure 11), treating the query as a single task, incorrectly extracts a distractor date (1985, the year of the shooting) and fails. In contrast, ReSeek’s trajectory (Figure 12) demonstrates robust multi-step reasoning. It first identifies the shooter (“Dennis Allen”), then uses the judge action to validate this intermediate result before initiating a second, focused search for the death year. This structured process prevents premature conclusions and leads to the correct answer where the baseline fails.

A.11.2. CASE STUDY II

The contrast between ReSeek’s success (Figure 14) and the baseline’s failure (Figure 13) on this second case study is even more telling. The baseline agent correctly identifies the creator’s name, “Loren Bouchard,” but its monolithic reasoning process stops there. Unable to find the birth date in the initial context and lacking a mechanism to initiate a follow-up query, it resorts to hallucinating an incorrect answer (“1985”) that is entirely absent from the evidence.

Table 16. Ablation on Embedding Choice. (Deterministic values)

Backbone	BM25	E5	Qwen	Conan	Avg.
qwen3b-base	19.1	28.8	29.0	28.8	28.9
qwen3b-ins	21.3	31.2	31.3	31.1	31.2
qwen7b-base	23.5	36.1	36.2	36.0	36.1
qwen7b-ins	25.0	37.7	38.1	37.9	37.9

 Table 17. Ablation on Reranker Choice (Mean $\pm$ Std).

Method	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	Musique	Bamboogle	FictionalHot	Avg.
None	0.391 $\pm$ 0.004	0.495 $\pm$ 0.005	0.362 $\pm$ 0.004	0.255 $\pm$ 0.006	0.218 $\pm$ 0.005	0.081 $\pm$ 0.003	0.243 $\pm$ 0.004	0.025 $\pm$ 0.002	0.259 $\pm$ 0.004
Regex	0.410 $\pm$ 0.003	0.541 $\pm$ 0.004	0.422 $\pm$ 0.005	0.320 $\pm$ 0.004	0.291 $\pm$ 0.003	0.093 $\pm$ 0.004	0.288 $\pm$ 0.005	0.042 $\pm$ 0.003	0.301 $\pm$ 0.003
Qwen	0.413 $\pm$ 0.004	0.557$\pm$0.003	0.432 $\pm$ 0.003	0.326 $\pm$ 0.005	0.301$\pm$0.004	0.101 $\pm$ 0.003	0.302 $\pm$ 0.004	0.057 $\pm$ 0.002	0.311 $\pm$ 0.003
ReSeek	0.415$\pm$0.003	0.553 $\pm$ 0.004	0.434$\pm$0.002	0.328$\pm$0.003	0.298 $\pm$ 0.005	0.103$\pm$0.002	0.304$\pm$0.003	0.059$\pm$0.002	0.312$\pm$0.002

In stark contrast, ReSeek demonstrates the power of structured self-correction. After its initial search, the judge action correctly identifies that the answer has not been found (`<judge> No </judge>`). This crucial validation step prompts the agent to formulate a new plan: use the newly found entity, “Loren Bouchard,” to perform a second, targeted search. This methodical decomposition of the problem allows ReSeek to navigate the multi-hop query, successfully retrieve the correct birth date, and avoid the pitfall of ungrounded generation that caused the baseline to fail.

A.12. Failure case study

To further investigate the concern regarding reward hacking and understand the performance plateau at higher turn limits, we analyze a representative failure case shown in Figure 15. In this episode, the user asked for a “deep water fishing boat with many baited hooks” (Ground Truth: *Longline fishing*). The maximum turn limit (T_{max}) was set to 10. As illustrated in the figure, the agent actively reformulated its search queries four times but failed to retrieve the target document, eventually providing an incorrect answer. Crucially, despite having a remaining budget of 6 turns, the agent voluntarily terminated the process at Turn 4.

This case highlights two key observations:

- **Corpus Limitations (Reason for Failure):** The failure was not due to a lack of effort or poor reasoning by the agent. The agent demonstrated effective *re-planning* capabilities, shifting its search strategy from the full description to specific keywords like “characteristics,” “multiple baited hooks,” and “design.” However, the retrieval system consistently returned irrelevant documents (e.g., *Trawlers*, *Dories*, and *Basnigs*), and the specific document for “Longline fishing” was not retrieved. This confirms that the performance plateau observed in our main experiments is primarily driven by information retrieval limits in the Wiki-18 corpus rather than agent inefficiency.
- **Evidence Against Reward Hacking:** This trajectory serves as strong empirical evidence that *reward hacking does not occur*. If the agent were motivated to artificially prolong the episode to “farm” rewards, it would be optimal to continue generating actions until Turn 10. Instead, the agent’s decision to stop early indicates that its learned policy correctly balances the potential gain of finding the answer against the penalty of additional steps. As seen in the final reasoning step (“I have exhausted reasonable search queries”), when the agent determines that further search is futile, it terminates the episode immediately to minimize the penalty.

Question: In what year did the man who shot the Chris Stockley, of The Dingoos, die?

Ground Truth: 1987

Search-R1:

<think> To find out in what year the man who shot the Chris Stockley, of The Dingoos, died, I will search for information about the incident and the man who shot Chris Stockley. </think>

<search> In what year did the man who shot the Chris Stockley, of The Dingoos, die? </search>

<information>

Doc 1(Title: "The Dingoos") and Tolhurst although Bois' book confirms that the song was written by Tolhurst alone. A week before the single was released Stockley received a serious gunshot wound during an incident at a party in Melbourne that resulted in a two-month stay in hospital, initially described as an 'accidental shooting', according to music historian, Ian McFarlane's "'Encyclopedia of Australian Rock and Pop'", Stockley was shot by notorious drug dealer Dennis Allen, who was trying to gate crash the party. An eight-hour benefit concert was held for Stockley on 4 November 1973 at Leggett's Ballroom

Doc 2(Title: "Dennis Allen (criminal)") in 1985. One victim who survived was guitarist Chris Stockley of The Dingoos, whom Allen shot in the stomach while attempting to gatecrash a party. Allen received a ten-year prison sentence for rape during the 1970s. It is also reported that he was a major drug dealer in the Richmond and South Yarra areas of Melbourne during the 1980s.

Doc 3(Title: "The Dingoos") The Dingoos The Dingoos is an Australian country rock band initially active from 1973 to 1979, formed in Melbourne which relocated to the United States from 1976. Most stable line-up was John Bois on bass guitar, John Lee on drums, Broderick Smith on vocals and harmonica, Chris Stockley on guitar, and Kerry Tolhurst on guitar. Mal

</information>

<answer>1985</answer>

Figure 11. A baseline agent (Search-R1) failing the two-hop question. The agent attempts to solve the problem in a single step and incorrectly extracts the year of the shooting (1985) instead of the correct year of death (1987).

Question: In what year did the man who shot the Chris Stockley, of The Dingoes, die?

Ground Truth: 1987

Ours:

<think> To find out the year the man who shot Chris Stockley, of The Dingoes, died, let's start by identifying the man who shot Chris Stockley. </think>

<search> Chris Stockley Dingoes shot</search>

<information>

Doc 1(Title: "The Dingoes") and Tolhurst although Bois' book confirms that the song was written by Tolhurst alone. A week before the single was released Stockley received a serious gunshot wound during an incident at a party in Melbourne that resulted in a two-month stay in hospital, initially described as an 'accidental shooting', according to music historian, Ian McFarlane's ""Encyclopedia of Australian Rock and Pop"", Stockley was shot by notorious drug dealer Dennis Allen, who was trying to gate crash the party.

Doc 2(Title: "The Dingoes") The Dingoes The Dingoes is an Australian country rock band initially active from 1973 to 1979, formed in Melbourne which relocated to the United States from 1976. Most stable line-up was John Bois on bass guitar, John Lee on drums, Broderick Smith on vocals and harmonica, Chris Stockley on guitar, and Kerry Tolhurst on guitar. Mal Logan (who provided keyboards on the first LP) on keyboards joined after Stockley was hospitalised when shot in the stomach by Melbourne drug-dealer,

Doc 3(Title: "Dennis Allen (criminal)") in 1985. One victim who survived was guitarist Chris Stockley of The Dingoes, whom Allen shot in the stomach while attempting to gatecrash a party. Allen received a ten-year prison sentence for rape during the 1970s. It is also reported that he was a major drug dealer in the Richmond and South Yarra areas of Melbourne during the 1980s. New South Wales Police Detective-Sergeant Roger Rogerson was convicted of supplying heroin in a deal with Allen, but was acquitted following appeal. Allen avoided capture and prosecution for his crimes by acting as an informant for the police. Allen died on

</information>

<judge>Yes</judge>

Based on the information provided, the man who shot Chris Stockley, of The Dingoes, is Dennis Allen. Now, let's find out the year he died.

<search>Garajonay National Park location</search>

<information>

Doc 1(Title: "Dennis Allen (criminal)") Dennis Allen (criminal) Dennis Bruce Allen (7 November 1951 – 13 April 1987) was an Australian drug dealer who was reported to have murdered many victims. He was based in Melbourne, and was the oldest son of criminal matriarch Kath Pettingill. Allen avoided capture for drug dealing by acting as an informant throughout his career. He died of heart disease in 1987

Doc 2(Title: "Dennis Allen (criminal)") 13 April 1987 of heart failure at St. Vincent's Hospital, Melbourne; ""pieces of his heart actually broke off after decades of heavy drug abuse"". His funeral was conducted by Father Peter Norden, a Jesuit priest who performed funerals for three members of the Pettingill family during the 1980s. Dennis Allen (criminal) Dennis Bruce Allen (7 November 1951 – 13 April 1987)

Doc 3(Title: "Dennis Allen (criminal)") in 1985. One victim who survived was guitarist Chris Stockley of The Dingoes, whom Allen shot in the stomach while attempting to gatecrash a party. Allen received a ten-year prison sentence for rape during the 1970s. It is also reported that he was a major drug dealer in the Richmond and South Yarra areas of Melbourne during the 1980s.

</information>

<judge>Yes</judge>

Based on the information provided, Dennis Allen died on 13 April 1987. Therefore, the year the man who shot Chris Stockley, of The Dingoes, died is 1987.

<answer>1987</answer>

Figure 12. A case study of ReSeek on a two-hop question. The agent first identifies the shooter ("Dennis Allen") and then finds his death year. The judge action is used to validate the intermediate finding before proceeding to the second reasoning step.

Question: When was the creator of Saddle Rash born?

Ground Truth: October 10, 1969

Search-R1:

`<think>` To find out when the creator of Saddle Rash was born, I will search for information about the creator.

`</think>`

`<search>` When was the creator of Saddle Rash born? `</search>`

`<information>`

Doc 1(Title: "Saddle Rash") Saddle Rash Saddle Rash is a canceled comedy animated series. The pilot episode was featured on March 24, 2002 on Cartoon Network's ""Adult Swim"" programming block. It was created by Loren Bouchard, co-creator of ""Home Movies"", and uses the same low-budget Flash animation technique found in seasons two and up of ""Home Movies"". ""Saddle Rash"" is based on a small town in the Old West, and narrated by an ""old prospector""-type chara

Doc 2(Title: "Saddle sore") Saddle sore A saddle sore in humans is a skin ailment on the buttocks due to, or exacerbated by, horse riding or cycling on a bicycle saddle. It often develops in three stages: skin abrasion, folliculitis (which looks like a small, reddish acne), and finally abscess. Because it most commonly starts with skin abrasion, it is desirable to reduce the factors

Doc 3(Title: "Saddle Rash") town for the purpose of revenge against Tommy Morgan (also voiced by Benjamin), who has been hiding in the hills due to a \$1000 bounty on his head. While Slim is in town, Hanna Headstrong (Sarah Silverman), the daughter of a rancher in town, falls in love with him. Eventually, Slim and Tommy Morgan stand off. ""All the members are listed in credited order.""

`</information>`

`<answer>`1985`</answer>`

Figure 13. A baseline agent (Search-R1) failing the two-hop question. While the agent’s search successfully identifies the creator, “Loren Bouchard,” it fails to perform the necessary follow-up search for their birth date. It prematurely concludes with a hallucinated and incorrect answer.

Question: When was the creator of Saddle Rash born?

Ground Truth: October 10, 1969

Search-R1:

<think> To find out when the creator of Saddle Rash was born, let's start by identifying who the creator is.

</think>

<search> creator of Saddle Rash birth date </search>

<information>

Doc 1(Title: "Saddle Rash") Saddle Rash Saddle Rash is a canceled comedy animated series. The pilot episode was featured on March 24, 2002 on Cartoon Network's ""Adult Swim"" programming block. It was created by Loren Bouchard, co-creator of ""Home Movies"", and uses the same low-budget Flash animation technique found in seasons two and up of ""Home Movies"". ""Saddle Rash"" is based on a small town in the Old West, and narrated by an ""old prospector""-type chara

Doc 2(Title: "Saddle sore") Saddle sore A saddle sore in humans is a skin ailment on the buttocks due to, or exacerbated by, horse riding or cycling on a bicycle saddle. It often develops in three stages: skin abrasion, folliculitis (which looks like a small, reddish acne), and finally abscess. Because it most commonly starts with skin abrasion, it is desirable to reduce the factors

Doc 3(Title: "Saddle Rash") town for the purpose of revenge against Tommy Morgan (also voiced by Benjamin), who has been hiding in the hills due to a \$1000 bounty on his head. While Slim is in town, Hanna Headstrong (Sarah Silverman), the daughter of a rancher in town, falls in love with him. Eventually, Slim and Tommy Morgan stand off. ""All the members are listed in credited order.""

</information>

<judge>No</judge>

The information provided does not directly answer the question about the birth date of the creator of Saddle Rash. Let's try another search.

<search> Loren Bouchard birth date </search>

<information>

Doc 1(Title: "Loren Bouchard") and the second began, critics began giving the series praise. The show has generally been viewed as a spiritual successor to ""King of the Hill"", which carried less emphasis on shock comedy and focused more on character-driven humor. A film adaptation based on the animated TV series is in the works and is scheduled for a July 17, 2020 release. Bouchard lives in Los Angeles with his wife, Holly Kretschmar. The couple married on September 3, 2006.

Doc 2(Title: "Sylvain Bouchard") Sylvain Bouchard Sylvain Bouchard (born April 12, 1970 in Loretteville, Quebec City, Canada) is a Canadian long track speed skater. He won the 1000m event at the 1998 World Single Distance Championships. He competed at the 1994 Winter Olympics, finishing 4th at the 500m event and 5th at the 1000m event. He also competed at the 1998 Winter Olympics, finishing 4th on 500m and 5th on 1000m. He earned the world record time in the 1000m event in 1995 and 1998.

Doc 3(Title: "Sylvain Bouchard") Sylvain Bouchard Sylvain Bouchard (born April 12, 1970 in Loretteville, Quebec City, Canada) is a Canadian long track speed skater. He won the 1000m event at the 1998 World Single Distance Championships. He competed at the 1994 Winter Olympics, finishing 4th at the 500m event and 5th at the 1000m event. He also competed at the 1998 Winter Olympics, finishing 4th on 500m and 5th on 1000m. He earned the world record time in the 1000m event in 1995 and 1998.

</information>

<judge>Yes</judge>

<answer> October 10, 1969 </answer>

Figure 14. A case study of ReSeek on a two-hop question. The agent first attempts a broad search but correctly uses the judge action to determine the retrieved information is insufficient. It then extracts the creator's name ("Loren Bouchard") from the initial context and initiates a second, focused search for the birth date.

	Question: deep water fishing boat with many baited hooks? Ground Truth: Longline Ours:
Turn 1	<think> I need to identify the type of fishing boat described. </think> <search> deep water fishing boat with many baited hooks </search> <information> Doc 1(Title: "Recreational boat fishing") spool reels are being used more and more. Baits are similar to those used for beach and rock fishing except they are of Doc 2(Title: "Fishing trawler") Fishing trawler A fishing trawler is a commercial fishing vessel designed to operate fishing trawls. Trawling is a method of fishing that Doc 3(Title: "Fishing vessel") a newspaper report at that time, the "'Crete'" had "'a deep cockpit, a chair fitted for landing big fish and leather pockets for placing the pole." </information> <judge>No</judge> The initial search results discussed recreational fishing and trawlers, which use nets, or specific individual vessels. I need to find information specifically describing the characteristics of commercial boats used for deep-water fishing to see if "many baited hooks" is a defining feature of a specific type.
Turn 2	<search> deep water fishing boat characteristics </search> <information> Doc 1(Title: Dory) Dory A dory is a small, shallow-draft boat, about long. Doc 2(Title: "Remote control fishing") fishing boat should be: - water tight up to a certain depth Doc 3(Title: "Recreational boat fishing") spool reels are being used more and more. Baits are similar to those used for beach and rock fishing except they are often larger since larger fish are targeted. </information> <judge>No</judge> The previous search focused on general characteristics and returned shallow water boats (Dory) or toys (RC boats). I need to refocus on the specific mechanism of "many baited hooks" combined with deep water to identify the vessel type.
Turn 3	<search> deep water fishing boat with multiple baited hooks </search> <information> Doc 1(Title: "Recreational boat fishing") spool reels are being used more and more. Baits are similar to those used for beach and rock fishing except they are often larger since larger fish are targeted. Doc 2(Title: "Fishing trawler") Fishing trawler A fishing trawler is a commercial fishing vessel designed to operate fishing trawls. Doc 3(Title: Basnig) stern of the boat, two on both sides on the first section of the boat, two on both sides on the midsection of the boat and two again on the last part of the boat each of the bamboos has a pulley and cable attached to the net with molded lead as weight [pabigat]. </information> <judge>No</judge> The results are still dominated by net fishing (trawlers, Basnig) or recreational methods. I will try searching for the design aspects of deep-water fishing boats to see if that leads to the correct terminology for a boat that deploys lines with hooks.
Turn 4	<search> deep water fishing boat design </search> <information> Doc 1(Title: "Remote control fishing") fishing boat should be: - water tight up to a certain depth Doc 2(Title: "Flat-bottomed boat") Flat-bottomed boat A flat-bottomed boat is a boat with a flat bottomed, two-chined hull, which allows it be used in shallow bodies of water, such as rivers, because it is less likely to ground. Doc 3(Title: "Jon boat") Jon boat A jon boat (or johnboat) is a flat-bottomed boat constructed of aluminum, fiberglass, or wood with one, two, or three bench seats. They are suitable for fishing and hunting. </information> <judge>No</judge> After several unsuccessful attempts to find the answer. Given the constraints of the task, I will provide an educated guess based on the information available. <answer>boat fishing</answer>

Figure 15. A case study illustrating a retrieval failure. The model attempts to identify the specific term “Longline” based on its functional description. Despite performing four rounds of iterative search and query reformulation (shifting focus from the full question to specific “characteristics” and “design”), the retrieval system consistently returns irrelevant documents regarding net-based fishing (e.g., “Trawler”) or shallow-water vessels (e.g., “Dory,” “Jon boat”). Consequently, lacking the necessary evidence in the retrieved context, the model is unable to derive the correct entity and resorts to an incorrect guess.